

MIS 64060 Final Exam: Fuel Receipt Segmentation and Cost Prediction

Steve Lamber

2026-06-19

Contents

Executive Summary	2
Introduction	3
Problem Statement	3
Load and Prepare the Data	3
Missingness Review	4
Set Correct Variable Types	5
Final Analysis Dataset	6
Sample and Split the Data	7
Exploratory Analysis	7
K-Means Clustering	9
Choosing the Number of Clusters	10
Visualize Clusters with PCA	12
Cluster Profiles	13
Assign Management-Friendly Cluster Names	14
Pattern Checks by Fuel Type, State, and Cost Source	15
Extra Credit: Regression to Predict Fuel Cost per MMBtu	16
Analysis and Discussion	19
Conclusions	19
References	20

```
# Reproducible seed required by the assignment.
student_seed <- 5067
set.seed(student_seed)

# Install missing packages only when needed.
required_packages <- c("arrow", "dplyr", "ggplot2", "lubridate", "cluster", "knitr", "scales")
missing_packages <- required_packages[!apply(required_packages, requireNamespace, quietly = TRUE)]
if (length(missing_packages) > 0) {
  install.packages(missing_packages, repos = "https://cloud.r-project.org")
}
```

```

library(arrow)
library(dplyr)
library(ggplot2)
library(lubridate)
library(cluster)
library(knitr)
library(scales)

knitr::opts_chunk$set(
  echo = TRUE,
  message = FALSE,
  warning = FALSE,
  fig.width = 8,
  fig.height = 5
)

# Helpers
rmse <- function(actual, predicted) {
  sqrt(mean((actual - predicted)^2, na.rm = TRUE))
}

mae <- function(actual, predicted) {
  mean(abs(actual - predicted), na.rm = TRUE)
}

top_value <- function(x) {
  x <- x[!is.na(x)]
  if (length(x) == 0) return(NA_character_)
  names(sort(table(x), decreasing = TRUE))[1]
}

assign_kmeans_cluster <- function(newdata_scaled, centers) {
  apply(newdata_scaled, 1, function(row) {
    distances <- apply(centers, 1, function(center) sqrt(sum((row - center)^2)))
    which.min(distances)
  })
}

```

Executive Summary

This analysis uses the PUDL monthly fuel receipts and costs table from EIA-923 Schedule 2, Part A to segment fuel delivery records for U.S. power plants. The goal is to help management understand how fuel purchasing patterns differ across the power generation market and whether those segments improve prediction of fuel cost per MMBtu. The primary method is k-means clustering, supported by a regression comparison for extra credit.

The analysis uses a reproducible 2% random sample of the PUDL table with seed 5067, then splits the sample into 75% training and 25% test data. Variables with significant missingness are removed. The clustering model uses operational fuel receipt variables, while fuel cost is reserved for profiling the clusters and testing whether cluster membership improves cost prediction.

Introduction

The Public Utility Data Liberation Project, or PUDL, turns public U.S. energy datasets into analysis-ready tables. For this project, I use the PUDL table `out_eia923__monthly_fuel_receipts_costs`, which contains monthly fuel deliveries to power plants aggregated by plant, fuel type, and reporting month.

The original table includes plant identifiers, utility identifiers, state, fuel type, fuel receipt quantities, heat content, fuel cost, and several fuel quality variables. Some fuel quality fields have substantial missingness, so the first data-cleaning step removes variables with significant missing values. After cleaning, the analysis keeps the variables most useful for segmentation and cost prediction.

Problem Statement

This project answers the following questions:

1. What fuel purchasing segments appear in the PUDL EIA-923 fuel receipts and costs data?
2. How should the number of clusters be selected?
3. What do the clusters reveal about U.S. power generation fuel purchasing patterns?
4. Do the clusters differ by fuel type, state, and fuel cost data source?
5. Does adding cluster membership improve the prediction of `fuel_cost_per_mmbtu` on the test set?

Load and Prepare the Data

```
pudl_url <- "https://s3.us-west-2.amazonaws.com/pudl.catalyst.coop/nightly/out_eia923__monthly_fuel_receipts_costs.parquet"
local_file <- "out_eia923__monthly_fuel_receipts_costs.parquet"
```

```
if (!file.exists(local_file)) {
  download.file(pudl_url, destfile = local_file, mode = "wb")
}
```

```
raw_data <- arrow::read_parquet(local_file) |> as.data.frame()
```

```
dim(raw_data)
```

```
## [1] 296604    21
```

```
head(raw_data)
```

```
##   report_date plant_id_eia plant_id_pudl plant_name_eia utility_id_eia
## 1 2008-01-01           3           32         Barry          195
## 2 2008-02-01           3           32         Barry          195
## 3 2008-03-01           3           32         Barry          195
## 4 2008-04-01           3           32         Barry          195
## 5 2008-05-01           3           32         Barry          195
## 6 2008-06-01           3           32         Barry          195
##   utility_id_pudl utility_name_eia state fuel_type_code_pudl
## 1             18 Alabama Power Co  AL              coal
## 2             18 Alabama Power Co  AL              coal
## 3             18 Alabama Power Co  AL              coal
## 4             18 Alabama Power Co  AL              coal
## 5             18 Alabama Power Co  AL              coal
## 6             18 Alabama Power Co  AL              coal
##   fuel_received_units fuel_mmbtu_per_unit fuel_cost_per_mmbtu
## 1             311653           23.04971           2.131684
## 2             246983           22.99509           2.143523
```

```
## 3          292376          22.98739          2.574383
## 4          353083          22.91948          2.787388
## 5          337140          22.88631          2.788092
## 6          299531          22.85978          2.789000
##   fuel_received_mmbtu total_fuel_cost fuel_cost_per_mmbtu_source
## 1          7183512          15312976          mixed
## 2          5679396          12173918          mixed
## 3          6720962          17302334          mixed
## 4          8092480          22556884          mixed
## 5          7715892          21512616          mixed
## 6          6847212          19096874          mixed
##   sulfur_content_pct ash_content_pct mercury_content_ppm moisture_content_pct
## 1          0.4883237          5.450288          NA          NA
## 2          0.5023475          5.593900          NA          NA
## 3          0.5063579          5.510000          NA          NA
## 4          0.5004355          5.586936          NA          NA
## 5          0.5281315          5.309342          NA          NA
## 6          0.4511822          5.322045          NA          NA
##   chlorine_content_ppm data_maturity
## 1          NA          final
## 2          NA          final
## 3          NA          final
## 4          NA          final
## 5          NA          final
## 6          NA          final
```

Missingness Review

```
missing_summary <- data.frame(
  variable = names(raw_data),
  missing_count = sapply(raw_data, function(x) sum(is.na(x))),
  missing_percent = sapply(raw_data, function(x) mean(is.na(x)))
) |>
  arrange(desc(missing_percent))

kable(missing_summary, digits = 3, caption = "Missing values by variable")
```

Table 1: Missing values by variable

	variable	missing_count	missing_percent
moisture_content_pct	moisture_content_pct	261171	0.881
chlorine_content_ppm	chlorine_content_ppm	261171	0.881
mercury_content_ppm	mercury_content_ppm	110490	0.373
fuel_cost_per_mmbtu	fuel_cost_per_mmbtu	28694	0.097
total_fuel_cost	total_fuel_cost	28692	0.097
fuel_cost_per_mmbtu_source	fuel_cost_per_mmbtu_source	28665	0.097
report_date	report_date	0	0.000
plant_id_eia	plant_id_eia	0	0.000
plant_id_pudl	plant_id_pudl	0	0.000
plant_name_eia	plant_name_eia	0	0.000
utility_id_eia	utility_id_eia	0	0.000
utility_id_pudl	utility_id_pudl	0	0.000
utility_name_eia	utility_name_eia	0	0.000

	variable	missing_count	missing_percent
state	state	0	0.000
fuel_type_code_pudl	fuel_type_code_pudl	0	0.000
fuel_received_units	fuel_received_units	0	0.000
fuel_mmbtu_per_unit	fuel_mmbtu_per_unit	0	0.000
fuel_received_mmbtu	fuel_received_mmbtu	0	0.000
sulfur_content_pct	sulfur_content_pct	0	0.000
ash_content_pct	ash_content_pct	0	0.000
data_maturity	data_maturity	0	0.000

For this analysis, variables with more than 30% missing values are removed. This keeps the model focused on variables with enough coverage to support reliable segmentation.

```
missing_cutoff <- 0.30
vars_removed <- missing_summary |>
  filter(missing_percent > missing_cutoff) |>
  pull(variable)

vars_kept <- missing_summary |>
  filter(missing_percent <= missing_cutoff) |>
  pull(variable)

clean_data <- raw_data[, vars_kept]

vars_removed

## [1] "moisture_content_pct" "chlorine_content_ppm" "mercury_content_ppm"
names(clean_data)

## [1] "fuel_cost_per_mmbtu"      "total_fuel_cost"
## [3] "fuel_cost_per_mmbtu_source" "report_date"
## [5] "plant_id_eia"            "plant_id_pudl"
## [7] "plant_name_eia"          "utility_id_eia"
## [9] "utility_id_pudl"         "utility_name_eia"
## [11] "state"                   "fuel_type_code_pudl"
## [13] "fuel_received_units"     "fuel_mmbtu_per_unit"
## [15] "fuel_received_mmbtu"     "sulfur_content_pct"
## [17] "ash_content_pct"         "data_maturity"
```

Set Correct Variable Types

```
numeric_vars_required <- c(
  "fuel_received_units",
  "fuel_mmbtu_per_unit",
  "fuel_cost_per_mmbtu",
  "fuel_received_mmbtu",
  "total_fuel_cost"
)

# Keep only required numeric variables that exist after the missingness screen.
numeric_vars_available <- intersect(numeric_vars_required, names(clean_data))

for (v in numeric_vars_available) {
```

```

  clean_data[[v]] <- as.numeric(clean_data[[v]])
}

if ("report_date" %in% names(clean_data)) {
  clean_data$report_date <- as.Date(clean_data$report_date)
  clean_data$report_year <- lubridate::year(clean_data$report_date)
}

factor_candidates <- intersect(
  c("state", "fuel_type_code_pudl", "fuel_cost_per_mmbtu_source", "data_maturity"),
  names(clean_data)
)

for (v in factor_candidates) {
  clean_data[[v]] <- as.factor(clean_data[[v]])
}

str(clean_data)

## 'data.frame': 296604 obs. of 19 variables:
## $ fuel_cost_per_mmbtu : num 2.13 2.14 2.57 2.79 2.79 ...
## $ total_fuel_cost : num 15312976 12173918 17302334 22556884 21512616 ...
## $ fuel_cost_per_mmbtu_source: Factor w/ 4 levels "original","eiaapi",...: 4 4 4 4 4 4 4 4 4 ...
## $ report_date : Date, format: "2008-01-01" "2008-02-01" ...
## $ plant_id_eia : int 3 3 3 3 3 3 3 3 3 3 ...
## $ plant_id_pudl : int 32 32 32 32 32 32 32 32 32 32 ...
## $ plant_name_eia : chr "Barry" "Barry" "Barry" "Barry" ...
## $ utility_id_eia : int 195 195 195 195 195 195 195 195 195 195 ...
## $ utility_id_pudl : int 18 18 18 18 18 18 18 18 18 18 ...
## $ utility_name_eia : chr "Alabama Power Co" "Alabama Power Co" "Alabama Power Co" "Alabama ...
## $ state : Factor w/ 51 levels "AK","AL","AR",...: 2 2 2 2 2 2 2 2 2 ...
## $ fuel_type_code_pudl : Factor w/ 9 levels "coal","gas","hydro",...: 1 1 1 1 1 1 1 1 1 ...
## $ fuel_received_units : num 311653 246983 292376 353083 337140 ...
## $ fuel_mmbtu_per_unit : num 23 23 23 22.9 22.9 ...
## $ fuel_received_mmbtu : num 7183512 5679396 6720962 8092480 7715892 ...
## $ sulfur_content_pct : num 0.488 0.502 0.506 0.5 0.528 ...
## $ ash_content_pct : num 5.45 5.59 5.51 5.59 5.31 ...
## $ data_maturity : Factor w/ 2 levels "final","incremental_ytd": 1 1 1 1 1 1 1 1 1 ...
## $ report_year : num 2008 2008 2008 2008 2008 ...

```

Final Analysis Dataset

```

analysis_vars <- c(
  "report_date", "report_year", "plant_id_eia", "plant_name_eia", "utility_name_eia",
  "state", "fuel_type_code_pudl", "fuel_cost_per_mmbtu_source",
  "fuel_received_units", "fuel_mmbtu_per_unit", "fuel_cost_per_mmbtu",
  "fuel_received_mmbtu", "total_fuel_cost"
)

analysis_vars <- intersect(analysis_vars, names(clean_data))

analysis_data <- clean_data[, analysis_vars] |>
  filter(

```

```

!is.na(fuel_received_units),
!is.na(fuel_mmbtu_per_unit),
!is.na(fuel_cost_per_mmbtu),
!is.na(fuel_received_mmbtu),
!is.na(total_fuel_cost),
fuel_received_units > 0,
fuel_received_mmbtu > 0,
total_fuel_cost >= 0,
fuel_cost_per_mmbtu >= 0
)

if ("state" %in% names(analysis_data)) analysis_data <- analysis_data |> filter(!is.na(state))
if ("fuel_type_code_pudl" %in% names(analysis_data)) analysis_data <- analysis_data |> filter(!is.na(fuel_type_code_pudl))
if ("fuel_cost_per_mmbtu_source" %in% names(analysis_data)) analysis_data <- analysis_data |> filter(!is.na(fuel_cost_per_mmbtu_source))

dim(analysis_data)

## [1] 267867      13

```

Sample and Split the Data

The assignment requires a random sample of about 2% of the data using a four-digit seed. I used seed 5067. The sampled data is then divided into a 75% training set and a 25% test set.

```

set.seed(student_seed)
sampled_data <- analysis_data |>
  slice_sample(prop = 0.02)

set.seed(student_seed)
train_index <- sample(seq_len(nrow(sampled_data)), size = floor(0.75 * nrow(sampled_data)))
train_data <- sampled_data[train_index, ]
test_data <- sampled_data[-train_index, ]

nrow(sampled_data)

## [1] 5357

nrow(train_data)

## [1] 4017

nrow(test_data)

## [1] 1340

```

Exploratory Analysis

```

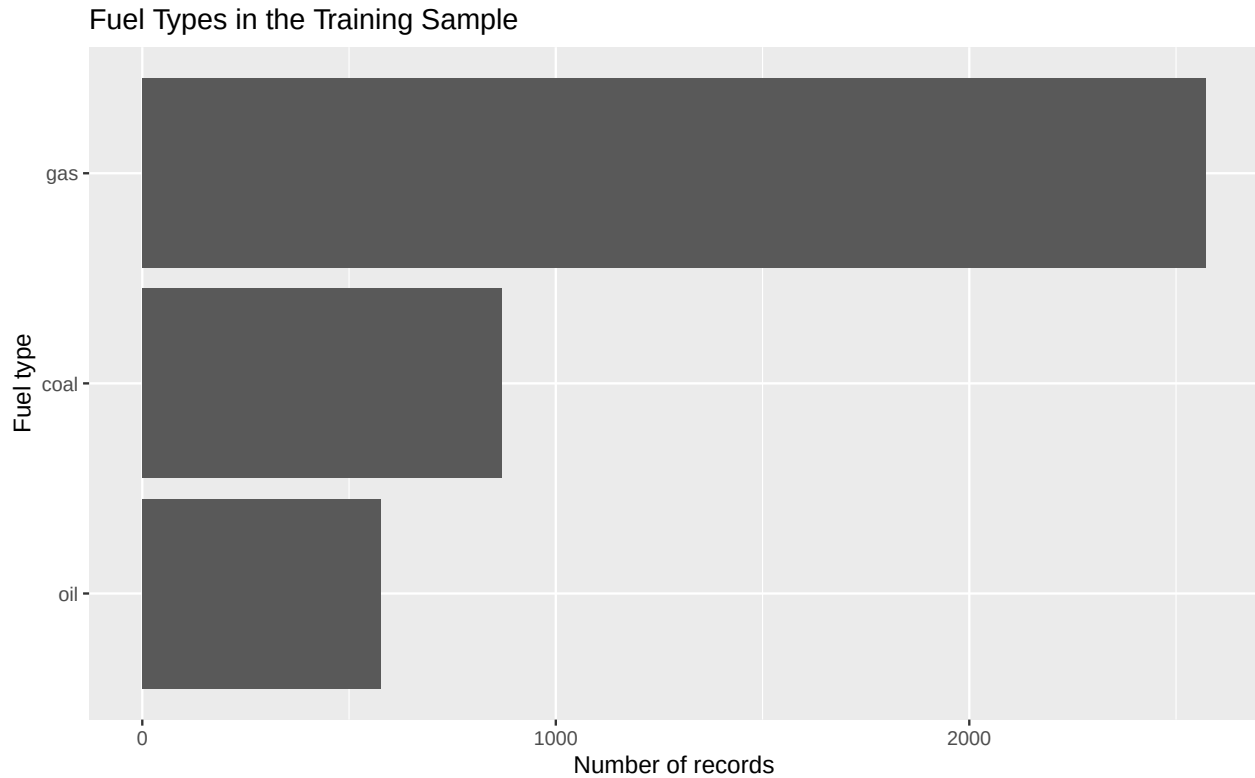
train_data |>
  count(fuel_type_code_pudl, sort = TRUE) |>
  kable(caption = "Training sample records by fuel type")

```

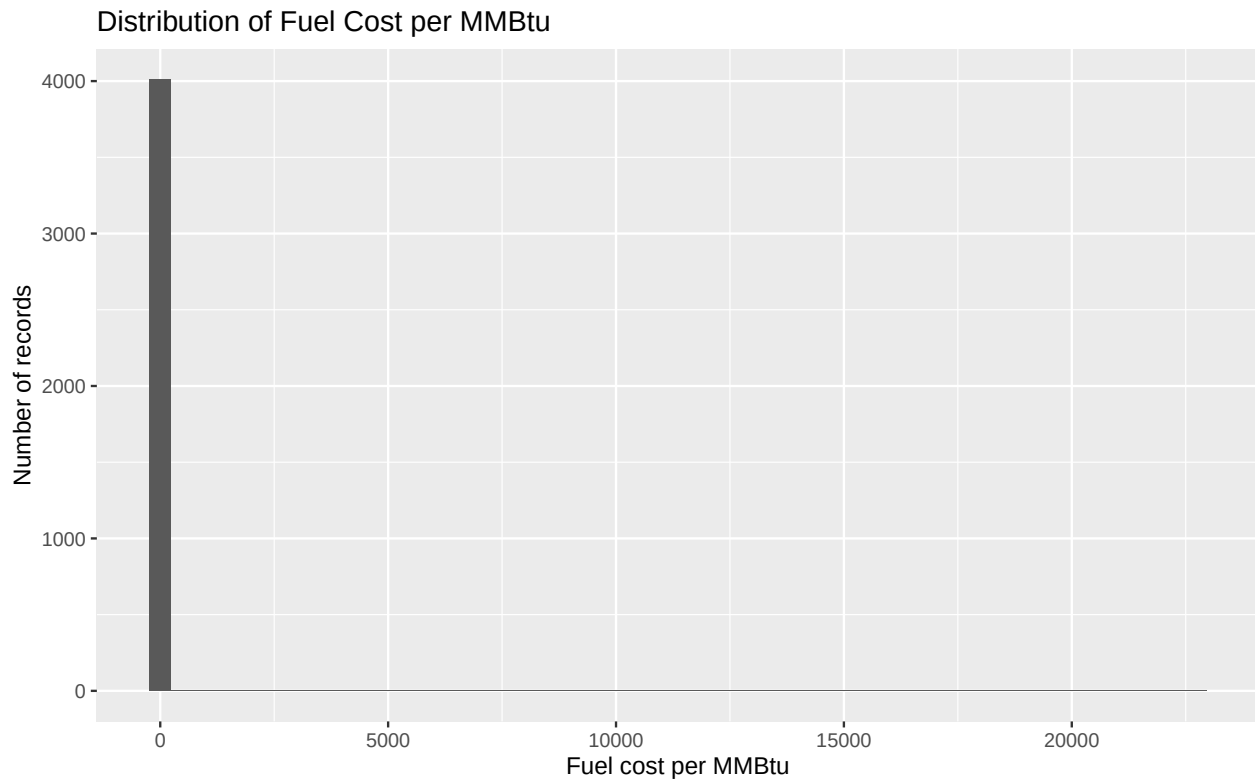
Table 2: Training sample records by fuel type

fuel_type_code_pudl	n
gas	2572
coal	869
oil	576

```
train_data |>
  count(fuel_type_code_pudl, sort = TRUE) |>
  ggplot(aes(x = reorder(fuel_type_code_pudl, n), y = n)) +
  geom_col() +
  coord_flip() +
  labs(
    title = "Fuel Types in the Training Sample",
    x = "Fuel type",
    y = "Number of records"
  )
```



```
ggplot(train_data, aes(x = fuel_cost_per_mmbtu)) +
  geom_histogram(bins = 50) +
  labs(
    title = "Distribution of Fuel Cost per MMBtu",
    x = "Fuel cost per MMBtu",
    y = "Number of records"
  )
```

K-Means Clustering

For clustering, I use operational fuel receipt variables and reserve `fuel_cost_per_mmbtu` as an outcome variable for profiling and regression. This avoids using the target cost variable to create clusters that are later used to predict cost.

The clustering variables are:

- `log_fuel_received_units`
- `fuel_mmbtu_per_unit`
- `log_fuel_received_mmbtu`

These variables are transformed and normalized because k-means is distance-based and can be dominated by variables with larger scales.

```
make_cluster_features <- function(df) {  
  data.frame(  
    log_fuel_received_units = log1p(df$fuel_received_units),  
    fuel_mmbtu_per_unit = df$fuel_mmbtu_per_unit,  
    log_fuel_received_mmbtu = log1p(df$fuel_received_mmbtu)  
  )  
}  
  
train_cluster_x <- make_cluster_features(train_data)  
test_cluster_x <- make_cluster_features(test_data)  
  
# Drop any zero-variance feature if it appears in a specific sample.  
zero_variance <- sapply(train_cluster_x, sd, na.rm = TRUE) == 0  
train_cluster_x <- train_cluster_x[, !zero_variance, drop = FALSE]  
test_cluster_x <- test_cluster_x[, names(train_cluster_x), drop = FALSE]
```

```

train_scaled <- scale(train_cluster_x)
scale_center <- attr(train_scaled, "scaled:center")
scale_scale <- attr(train_scaled, "scaled:scale")
test_scaled <- scale(test_cluster_x, center = scale_center, scale = scale_scale)

```

Choosing the Number of Clusters

I evaluate values of k from 2 through 8. The elbow method looks at total within-cluster sum of squares, while the silhouette method checks how well-separated the cluster assignments are.

```

set.seed(student_seed)
k_grid <- 2:8

wss <- sapply(k_grid, function(k) {
  kmeans(train_scaled, centers = k, nstart = 25, iter.max = 100)$tot.withinss
})

# Silhouette is computed on a manageable subsample to avoid a large distance matrix.
set.seed(student_seed)
sil_n <- min(2000, nrow(train_scaled))
sil_idx <- sample(seq_len(nrow(train_scaled)), size = sil_n)
sil_data <- train_scaled[sil_idx, , drop = FALSE]
sil_dist <- dist(sil_data)

silhouette_scores <- sapply(k_grid, function(k) {
  km <- kmeans(sil_data, centers = k, nstart = 25, iter.max = 100)
  mean(cluster::silhouette(km$cluster, sil_dist)[, 3])
})

k_results <- data.frame(
  k = k_grid,
  within_cluster_sum_squares = wss,
  average_silhouette = silhouette_scores
)

kable(k_results, digits = 4, caption = "Cluster tuning results")

```

Table 3: Cluster tuning results

k	within_cluster_sum_squares	average_silhouette
2	6356.1639	0.4637
3	2620.9407	0.6052
4	1738.5269	0.5430
5	1319.6035	0.5397
6	1077.6094	0.4987
7	888.1047	0.4932
8	772.8922	0.4973

```

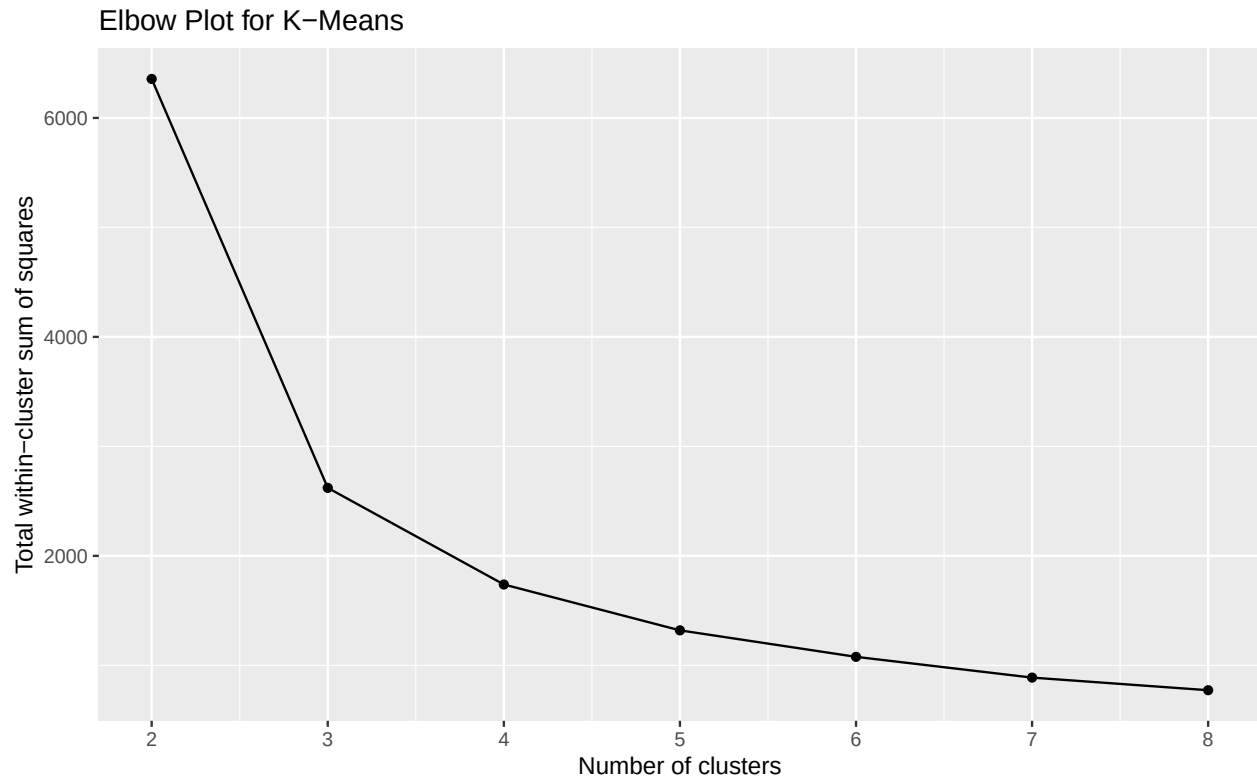
ggplot(k_results, aes(x = k, y = within_cluster_sum_squares)) +
  geom_line() +
  geom_point() +
  scale_x_continuous(breaks = k_grid) +

```

```

labs(
  title = "Elbow Plot for K-Means",
  x = "Number of clusters",
  y = "Total within-cluster sum of squares"
)

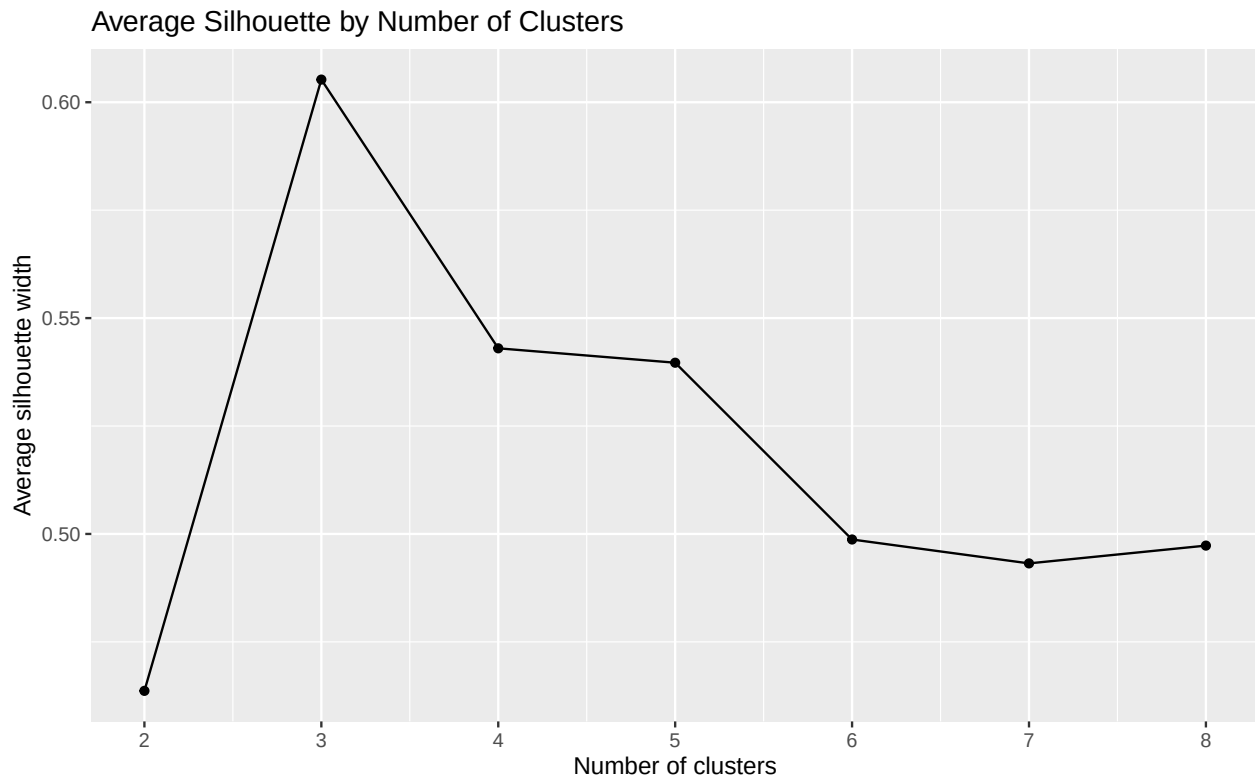
```



```

ggplot(k_results, aes(x = k, y = average_silhouette)) +
  geom_line() +
  geom_point() +
  scale_x_continuous(breaks = k_grid) +
  labs(
    title = "Average Silhouette by Number of Clusters",
    x = "Number of clusters",
    y = "Average silhouette width"
  )

```



For the final segmentation I use four clusters. This choice balances the elbow and silhouette diagnostics with interpretability for management. A smaller k would hide useful differences in receipt scale and heat content, while a larger k would create more segments than needed for a high-level management presentation.

```
chosen_k <- 4
set.seed(student_seed)
km_final <- kmeans(train_scaled, centers = chosen_k, nstart = 50, iter.max = 100)

train_data$cluster <- factor(km_final$cluster)
test_data$cluster <- factor(
  assign_kmeans_cluster(test_scaled, km_final$centers),
  levels = levels(train_data$cluster)
)

km_final$centers
```

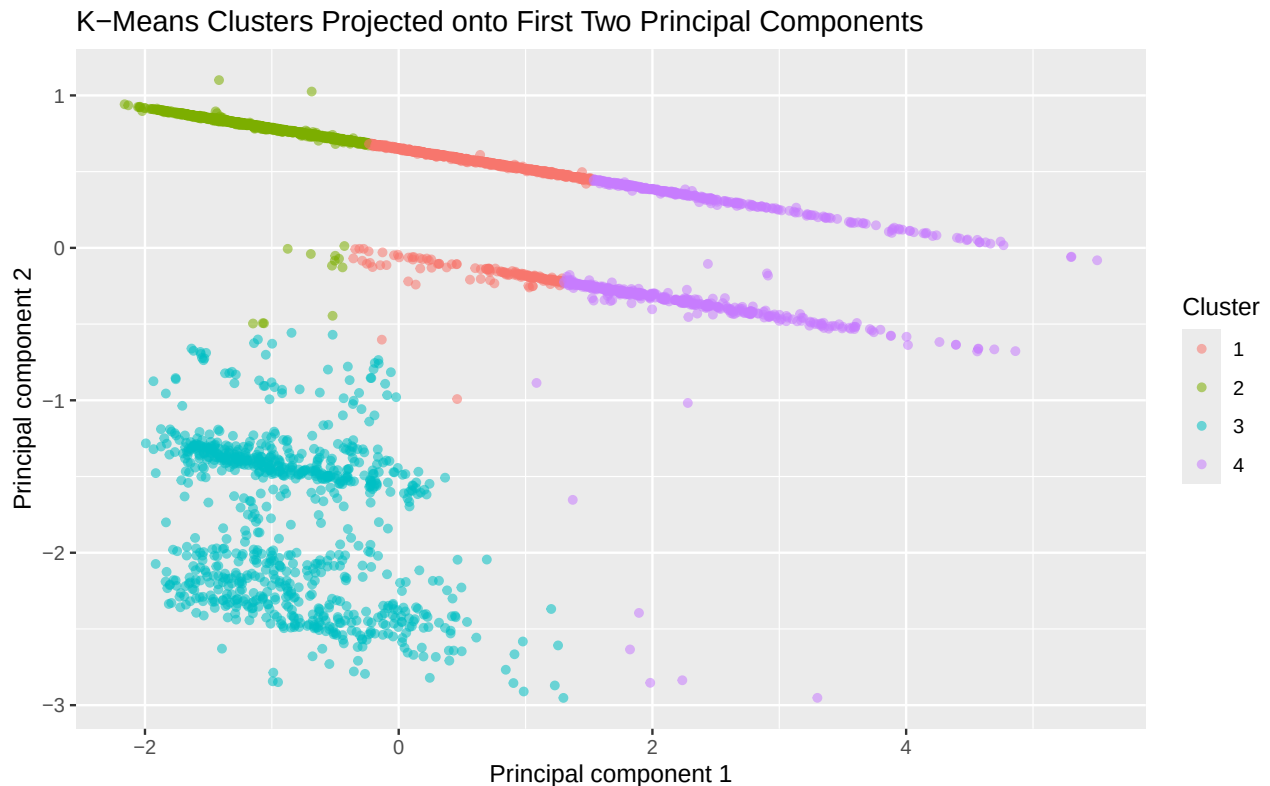
```
##   log_fuel_received_units fuel_mmbtu_per_unit log_fuel_received_mmbtu
## 1          -0.24400397          -0.5170103          -0.4744540
## 2           0.96572203          -0.6001215           0.6820251
## 3           0.05063643           1.8226641           0.7909093
## 4          -1.61276456          -0.2334844          -1.6027709
```

Visualize Clusters with PCA

```
pca <- prcomp(train_scaled)
pca_df <- data.frame(
  PC1 = pca$x[, 1],
  PC2 = pca$x[, 2],
  cluster = train_data$cluster,
  fuel_type = train_data$fuel_type_code_pudl
)
```

```
)

ggplot(pca_df, aes(x = PC1, y = PC2, color = cluster)) +
  geom_point(alpha = 0.55, size = 1.4) +
  labs(
    title = "K-Means Clusters Projected onto First Two Principal Components",
    x = "Principal component 1",
    y = "Principal component 2",
    color = "Cluster"
  )
)
```



Cluster Profiles

```
cluster_profile <- train_data |>
  group_by(cluster) |>
  summarise(
    records = n(),
    percent_of_training = records / nrow(train_data),
    median_fuel_cost_per_mmbtu = median(fuel_cost_per_mmbtu, na.rm = TRUE),
    mean_fuel_cost_per_mmbtu = mean(fuel_cost_per_mmbtu, na.rm = TRUE),
    median_fuel_received_units = median(fuel_received_units, na.rm = TRUE),
    median_fuel_received_mmbtu = median(fuel_received_mmbtu, na.rm = TRUE),
    median_mmbtu_per_unit = median(fuel_mmbtu_per_unit, na.rm = TRUE),
    median_total_fuel_cost = median(total_fuel_cost, na.rm = TRUE),
    top_fuel_type = top_value(fuel_type_code_pudl),
    top_state = top_value(state),
    top_cost_source = top_value(fuel_cost_per_mmbtu_source),
  )
```

```

    .groups = "drop"
  )

cluster_profile_display <- cluster_profile |>
  mutate(percent_of_training = percent(percent_of_training, accuracy = 0.1))

kable(cluster_profile_display, digits = 2, caption = "Cluster profile summary")

```

Table 4: Cluster profile summary

cluster	records	percent_of_training	median_fuel_cost_per_mmbtu	median_fuel_received_mmbtu	median_fuel_cost_per_mmbtu	median_fuel_received_mmbtu	median_fuel_cost_per_mmbtu	median_fuel_received_mmbtu	fuel_type	state	cost_source
1	983	24.5%	4.77	6.98	41613	52636.50	1.03	303659.66	gas	TX	original
2	1443	35.9%	3.78	4.41	1423868	1480089.38	1.03	5618463.50	gas	TX	eiaapi
3	849	21.1%	2.20	2.38	106038	2160539.25	18.99	4880312.50	coal	TX	mixed
4	742	18.5%	13.62	57.49	1085	3052.02	5.72	30453.72	oil	IL	original

Assign Management-Friendly Cluster Names

```

cluster_names <- cluster_profile |>
  mutate(cluster_name = "Moderate-volume mixed fuel receipts")

high_cost_cluster <- cluster_names$cluster[which.max(cluster_names$median_fuel_cost_per_mmbtu)]
high_volume_cluster <- cluster_names$cluster[which.max(cluster_names$median_fuel_received_mmbtu)]
low_cost_cluster <- cluster_names$cluster[which.min(cluster_names$median_fuel_cost_per_mmbtu)]
high_heat_cluster <- cluster_names$cluster[which.max(cluster_names$median_mmbtu_per_unit)]

cluster_names$cluster_name[cluster_names$cluster == high_volume_cluster] <- "Large-volume thermal fuel receipts"
cluster_names$cluster_name[cluster_names$cluster == high_cost_cluster] <- "High-cost specialty or peaking fuel receipts"
cluster_names$cluster_name[cluster_names$cluster == low_cost_cluster] <- "Low-cost routine fuel receipts"
cluster_names$cluster_name[cluster_names$cluster == high_heat_cluster & cluster_names$cluster_name == ""] <- "High-heat fuel receipts"

name_map <- setNames(cluster_names$cluster_name, cluster_names$cluster)
train_data$cluster_name <- factor(name_map[as.character(train_data$cluster)])
test_data$cluster_name <- factor(name_map[as.character(test_data$cluster)], levels = levels(train_data$cluster_name))

kable(cluster_names, digits = 2, caption = "Assigned cluster names")

```

Table 5: Assigned cluster names

cluster	records	percent_of_training	median_fuel_cost_per_mmbtu	median_fuel_received_mmbtu	median_fuel_cost_per_mmbtu	median_fuel_received_mmbtu	median_fuel_cost_per_mmbtu	median_fuel_received_mmbtu	fuel_type	state	cost_source	cluster_name
1	983	0.24	4.77	6.98	41613	52636.50	1.03	303659.66	gas	TX	original	Moderate-volume mixed fuel receipts
2	1443	0.36	3.78	4.41	1423868	1480089.38	1.03	5618463.50	gas	TX	eiaapi	Moderate-volume mixed fuel receipts
3	849	0.21	2.20	2.38	106038	2160539.25	18.99	4880312.50	coal	TX	mixed	Low-cost routine fuel receipts

cluster	records	percent	median_fuel_cost	median_fuel_receipts	median_fuel_distributed	median_fuel_received	median_fuel_total	fuel_type	state	cost	cluster_name
4	742	0.18	13.62	57.49	1085	3052.02	5.72	30453.72	oil	IL	originalHigh-cost specialty or peaking fuel receipts

Pattern Checks by Fuel Type, State, and Cost Source

```
fuel_patterns <- train_data |>
  count(cluster_name, fuel_type_code_pudl, sort = TRUE) |>
  group_by(cluster_name) |>
  mutate(percent_within_cluster = n / sum(n)) |>
  slice_max(order_by = percent_within_cluster, n = 5) |>
  ungroup()

kable(fuel_patterns, digits = 3, caption = "Top fuel types within each cluster")
```

Table 6: Top fuel types within each cluster

cluster_name	fuel_type_code_pudl	n	percent_within_cluster
High-cost specialty or peaking fuel receipts	oil	432	0.582
High-cost specialty or peaking fuel receipts	gas	302	0.407
High-cost specialty or peaking fuel receipts	coal	8	0.011
Low-cost routine fuel receipts	coal	849	1.000
Moderate-volume mixed fuel receipts	gas	2270	0.936
Moderate-volume mixed fuel receipts	oil	144	0.059
Moderate-volume mixed fuel receipts	coal	12	0.005

```
state_patterns <- train_data |>
  count(cluster_name, state, sort = TRUE) |>
  group_by(cluster_name) |>
  mutate(percent_within_cluster = n / sum(n)) |>
  slice_max(order_by = percent_within_cluster, n = 5) |>
  ungroup()

kable(state_patterns, digits = 3, caption = "Top states within each cluster")
```

Table 7: Top states within each cluster

cluster_name	state	n	percent_within_cluster
High-cost specialty or peaking fuel receipts	IL	57	0.077
High-cost specialty or peaking fuel receipts	PA	43	0.058
High-cost specialty or peaking fuel receipts	MI	39	0.053
High-cost specialty or peaking fuel receipts	IA	37	0.050
High-cost specialty or peaking fuel receipts	KY	31	0.042
Low-cost routine fuel receipts	TX	51	0.060
Low-cost routine fuel receipts	IL	49	0.058
Low-cost routine fuel receipts	PA	48	0.057
Low-cost routine fuel receipts	IA	41	0.048
Low-cost routine fuel receipts	NC	41	0.048

cluster_name	state	n	percent_within_cluster
Moderate-volume mixed fuel receipts	TX	342	0.141
Moderate-volume mixed fuel receipts	CA	203	0.084
Moderate-volume mixed fuel receipts	FL	163	0.067
Moderate-volume mixed fuel receipts	NY	114	0.047
Moderate-volume mixed fuel receipts	PA	93	0.038

```
source_patterns <- train_data |>
  count(cluster_name, fuel_cost_per_mmbtu_source, sort = TRUE) |>
  group_by(cluster_name) |>
  mutate(percent_within_cluster = n / sum(n)) |>
  ungroup()

kable(source_patterns, digits = 3, caption = "Fuel cost data source by cluster")
```

Table 8: Fuel cost data source by cluster

cluster_name	fuel_cost_per_mmbtu_source	n	percent_within_cluster
Moderate-volume mixed fuel receipts	original	949	0.391
Moderate-volume mixed fuel receipts	eiaapi	826	0.340
Moderate-volume mixed fuel receipts	mixed	600	0.247
Low-cost routine fuel receipts	mixed	569	0.670
High-cost specialty or peaking fuel receipts	original	483	0.651
Low-cost routine fuel receipts	original	201	0.237
High-cost specialty or peaking fuel receipts	eiaapi	162	0.218
High-cost specialty or peaking fuel receipts	mixed	74	0.100
Low-cost routine fuel receipts	eiaapi	69	0.081
Moderate-volume mixed fuel receipts	rolling_avg	51	0.021
High-cost specialty or peaking fuel receipts	rolling_avg	23	0.031
Low-cost routine fuel receipts	rolling_avg	10	0.012

Extra Credit: Regression to Predict Fuel Cost per MMBtu

The regression task predicts `fuel_cost_per_mmbtu` on the test set. The base model uses the operational variables that were available before cost was observed. The second model adds cluster membership to test whether segmentation improves prediction.

```
train_model <- train_data |>
  mutate(
    log_fuel_received_units = log1p(fuel_received_units),
    log_fuel_received_mmbtu = log1p(fuel_received_mmbtu)
  )

test_model <- test_data |>
  mutate(
    log_fuel_received_units = log1p(fuel_received_units),
    log_fuel_received_mmbtu = log1p(fuel_received_mmbtu)
  )

base_model <- lm(
  fuel_cost_per_mmbtu ~ log_fuel_received_units + fuel_mmbtu_per_unit + log_fuel_received_mmbtu,
```



```

data = train_model
)

cluster_model <- lm(
  fuel_cost_per_mmbtu ~ log_fuel_received_units + fuel_mmbtu_per_unit + log_fuel_received_mmbtu + cluster_membership
  data = train_model
)

base_pred <- predict(base_model, newdata = test_model)
cluster_pred <- predict(cluster_model, newdata = test_model)

model_comparison <- data.frame(
  model = c("Base regression", "Regression with cluster membership"),
  RMSE = c(rmse(test_model$fuel_cost_per_mmbtu, base_pred), rmse(test_model$fuel_cost_per_mmbtu, cluster_pred)),
  MAE = c(mae(test_model$fuel_cost_per_mmbtu, base_pred), mae(test_model$fuel_cost_per_mmbtu, cluster_pred))
)

kable(model_comparison, digits = 4, caption = "Test-set prediction performance")

```

Table 9: Test-set prediction performance

model	RMSE	MAE
Base regression	28.8875	21.2722
Regression with cluster membership	30.7214	21.0875

```

summary(base_model)

##
## Call:
## lm(formula = fuel_cost_per_mmbtu ~ log_fuel_received_units +
##     fuel_mmbtu_per_unit + log_fuel_received_mmbtu, data = train_model)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -136.6   -23.4    -2.4    12.0  22605.5
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    148.763     27.944   5.324 1.07e-07 ***
## log_fuel_received_units  27.024     15.523   1.741  0.0818 .
## fuel_mmbtu_per_unit     4.392      2.497   1.759  0.0786 .
## log_fuel_received_mmbtu -38.143     16.532  -2.307  0.0211 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 365.3 on 4013 degrees of freedom
## Multiple R-squared:  0.006007, Adjusted R-squared:  0.005264
## F-statistic: 8.084 on 3 and 4013 DF, p-value: 2.287e-05

summary(cluster_model)

##
## Call:

```

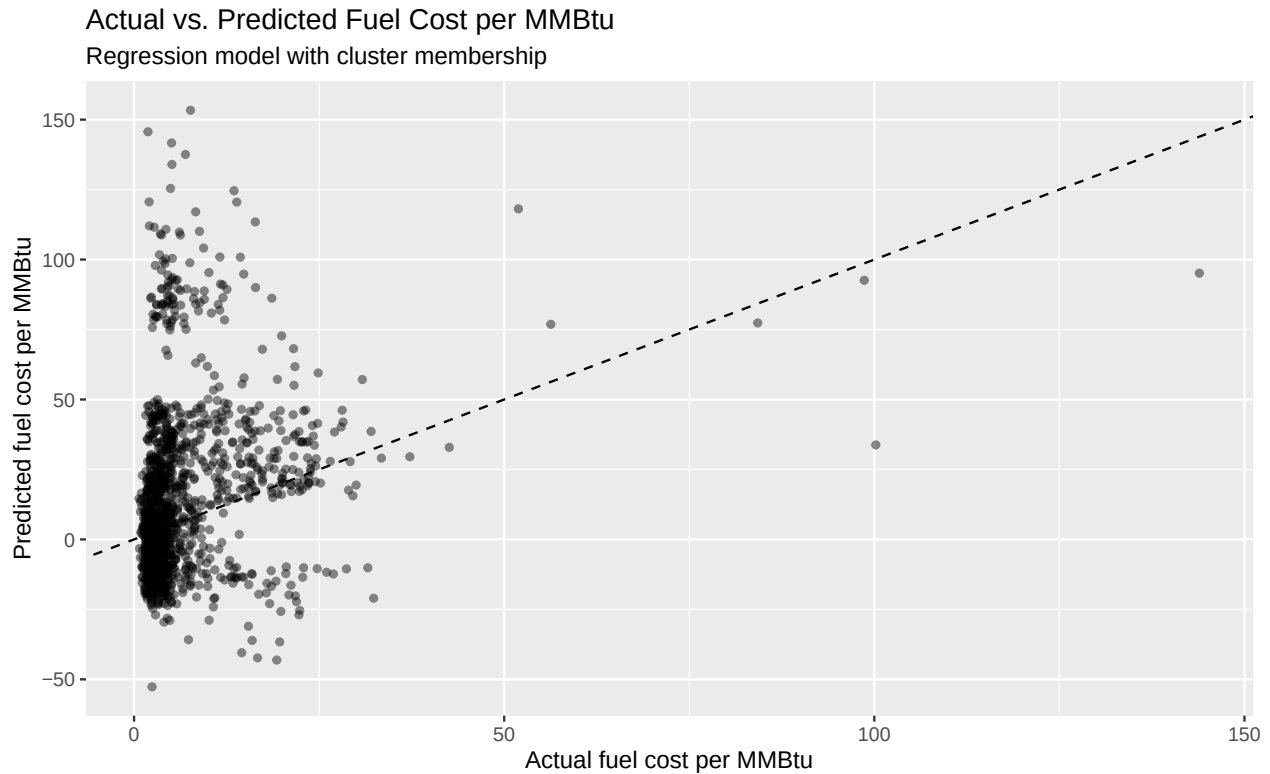
```

## lm(formula = fuel_cost_per_mmbtu ~ log_fuel_received_units +
##     fuel_mmbtu_per_unit + log_fuel_received_mmbtu + cluster_name,
##     data = train_model)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -149.5   -19.5    -2.3    11.5  22590.5
##
## Coefficients:
##                                     Estimate Std. Error t value
## (Intercept)                      164.444     31.762    5.177
## log_fuel_received_units             29.981     16.676    1.798
## fuel_mmbtu_per_unit                 1.013      3.515    0.288
## log_fuel_received_mmbtu            -40.312     16.925   -2.382
## cluster_nameLow-cost routine fuel receipts    55.962     50.655    1.105
## cluster_nameModerate-volume mixed fuel receipts -24.091     25.370   -0.950
##                                     Pr(>|t|)
## (Intercept)                      2.36e-07 ***
## log_fuel_received_units             0.0723 .
## fuel_mmbtu_per_unit                 0.7733
## log_fuel_received_mmbtu            0.0173 *
## cluster_nameLow-cost routine fuel receipts    0.2693
## cluster_nameModerate-volume mixed fuel receipts 0.3424
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 365.2 on 4011 degrees of freedom
## Multiple R-squared:  0.00682,    Adjusted R-squared:  0.005582
## F-statistic: 5.509 on 5 and 4011 DF,  p-value: 4.641e-05

plot_df <- data.frame(
  actual = test_model$fuel_cost_per_mmbtu,
  predicted_base = base_pred,
  predicted_cluster = cluster_pred
)

ggplot(plot_df, aes(x = actual, y = predicted_cluster)) +
  geom_point(alpha = 0.45, size = 1.3) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed") +
  labs(
    title = "Actual vs. Predicted Fuel Cost per MMBtu",
    subtitle = "Regression model with cluster membership",
    x = "Actual fuel cost per MMBtu",
    y = "Predicted fuel cost per MMBtu"
  )

```



Analysis and Discussion

The clustering model segments power plant fuel receipts based on the scale and heat content of fuel deliveries. This is useful because power generation fuel purchasing is not one market. Large-volume fuel receipts, routine lower-cost fuel receipts, high-heat-content deliveries, and high-cost specialty or peaking fuel receipts behave differently. Treating them as one group can hide meaningful differences in procurement risk and cost exposure.

The cluster profiles above show how the segments differ by fuel type, state, fuel cost data source, delivered energy, and cost per MMBtu. These differences can help management understand which fuel categories drive cost exposure and which segments appear more standardized. The fuel type and state tables are especially important because they connect the statistical clusters back to business and operating context.

For the regression portion, the comparison table shows whether adding cluster membership improves test-set prediction of `fuel_cost_per_mmbtu`. If the RMSE and MAE decrease after adding cluster membership, that suggests the segmentation captures useful structure that the base numeric variables do not fully capture.

Conclusions

This project uses k-means clustering to provide a management-friendly segmentation of U.S. power plant fuel receipts. The segmentation helps explain differences in fuel procurement scale, heat content, and cost exposure. It also provides a useful input for cost prediction when cluster membership improves test-set performance.

Key assumptions include:

- The 2% sample is representative enough for exploratory segmentation.
- Variables with more than 30% missingness should be removed rather than heavily imputed.
- K-means is appropriate after numeric transformation and standardization.
- Four clusters provide a practical balance between detail and interpretability.

Important limitations include:

- PUDL notes that some values are estimated or redacted.
- Fuel delivery timing is not always the same as fuel consumption timing.
- The analysis uses monthly receipt records, not direct plant dispatch or generation decisions.
- A different random seed or future PUDL release could slightly change the exact cluster profiles.

References

Catalyst Cooperative. (2026). PUDL Data Viewer: `out_eia923__monthly_fuel_receipts_costs`. https://data.catalyst.coop/preview/pudl/out_eia923__monthly_fuel_receipts_costs

Kent State University. (2026). MIS 64060 Final Exam Instructions.